C# Operators and Flow Control

Operators and flow-control statements form the foundation of program logic. **Operators** perform calculations, comparisons, assignments, and logical operations, while **flow-control statements** determine which statements execute and how often they execute.

A useful mental model is:

Data
↓
Operators
↓
Conditions
↓
Flow Control
↓
Program Behavior

The topics naturally divide into two parts: **operators** and **decision/iteration structures**.

---

1. C# Operators

An **operator** is a symbol or keyword that performs an operation on one or more operands.

For example:

```
int total = price * quantity;
```

Here:

- `price` and `quantity` are operands.
- `*` is the multiplication operator.
- `=` is the assignment operator.

Understanding operators is particularly important because they are used inside conditions, calculations, loops, and assignments.

---

2. Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Operation	Example

`+`	Addition	`a + b`
`-`	Subtraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division	`a / b`
`%`	Modulo/remainder	`a % b`
`-`	Unary negation	`-a`

Example

```
int price = 20;

int quantity = 3;


int subtotal = price * quantity;

int remainder = 17 % 5;


Console.WriteLine($"Subtotal: {subtotal}");

Console.WriteLine($"Remainder: {remainder}");
```

Output:

```
Subtotal: 60

Remainder: 2
```

The `%` operator returns the **remainder** after division. It is especially useful when determining whether a number is even or odd:

```
int number = 14;


bool isEven = number % 2 == 0;
```

Tip — Modulo

`%` is frequently used for periodic logic, divisibility tests, and even/odd checks.

---

3. Compound Assignment Operators

Compound assignment combines an operation and an assignment.

Instead of:

```
total = total + 10;
```

you can write:

```
total += 10;
```

Common compound assignment operators are:

Operator	Equivalent
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x *= y`	`x = x * y`
`x /= y`	`x = x / y`
`x %= y`	`x = x % y`
`x &= y`	`x = x & y`
`x |= y`	`x = x | y`
`x ^= y`	`x = x ^ y`

Practical Example

```
decimal balance = 100m;

balance += 50m;

balance -= 20m;

Console.WriteLine(balance);
```

Result:

```
130
```

The important point is that the operation modifies the existing variable.

---

4. Relational Operators

Relational operators compare two values and produce a **`bool`** result: **`true`** or **`false`**.

Operator	Meaning
`==`	Equal
`!=`	Not equal
`>`	Greater than
`>=`	Greater than or equal
`<`	Less than
`<=`	Less than or equal

Example:

```
int age = 20;
Console.WriteLine(age >= 18);
```

Output:

```
True
```

Relational operators become particularly useful when combined with flow control:

```
if (age >= 18)
{
    Console.WriteLine("Adult");
}
```

Pitfall — = vs. ==

= performs **assignment**.

== performs an **equality comparison**.

These operations have completely different purposes.

---

5. Boolean Logical Operators

Logical operators combine or modify Boolean expressions.

The principal operators are:

Operator	Meaning
`!x`	NOT — reverses `x`
`x && y`	AND — true when both are true
`x || y`	OR — true when at least one is true
`x ^ y`	XOR — true when exactly one is true

For example:

```
int age = 21;
bool hasMembership = true;
if (age >= 18 && hasMembership)
{
    Console.WriteLine("Access granted.");
}
```

Both conditions must be true.

Logical OR

```
bool isStudent = false;
bool isSenior = true;

if (isStudent || isSenior)
{
    Console.WriteLine("Discount available.");
}
```

Only one condition needs to be true.

Logical NOT

```
bool isBlocked = false;

if (!isBlocked)
{
```

```
        Console.WriteLine("Access allowed.");
    }
```

`!false` becomes `true`.

---

6. Short-Circuit Evaluation

There is an important distinction between:

`&` `|`

and:

`&&` `||`

With Boolean operands, `&` and `|` evaluate **both operands**. By contrast, `&&` and `||` evaluate the right-hand operand only when necessary.

Consider:

```
    if (age >= 18 && hasMembership)
    {
        Console.WriteLine("Access granted.");
    }
```

If:

`age >= 18`

is already `false`, the entire AND expression must be false. Therefore, C# does not need to evaluate the second operand.

This is called **short-circuit evaluation**.

Note

For integral numeric operands, `&`, `|`, and `^` perform **bitwise operations**, rather than Boolean short-circuit operations.

---

7. Other Operators

C# includes several additional operators. The material identifies `sizeof`, `typeof`, the conditional operator `?:`, `is`, `as`, and pointer-related operators.

For this section, the most directly useful examples are:

typeof

Returns information representing a type:

```
Type type = typeof(string);


Console.WriteLine(type);
```

is

Determines whether an object is compatible with a particular type:

```
    object value = "Hello";


    if (value is string)
    {
        Console.WriteLine("The object is a string.");
    }
```

as

Attempts a compatible reference conversion:

```
object value = "Hello";

string text = value as string;
```

If the conversion is not possible, **as** produces **null** rather than performing a successful conversion.

Conditional Operator ?:

The conditional operator provides a compact conditional expression:

```
int age = 20;

string category = age >= 18 ? "Adult" : "Minor";
```

Conceptually:

```
condition ? value-if-true : value-if-false
```

Note

The source also lists pointer operators `&` and `*`. They belong to specialized pointer-related usage; the important point here is recognizing that operator meaning can depend on context and operand types.

---

8. Operator Precedence

When an expression contains several operators, **operator precedence** determines which operation is evaluated first. The precedence table places unary operations near the highest level, followed by multiplication/division/modulo, addition/subtraction, comparisons, equality, logical operations, conditional operations, and finally assignment operators.

Consider:

```
int result = 10 + 5 * 2;
```

Multiplication has higher precedence:

```
10 + (5 * 2)
```

```
10 + 10
```

```
20
```

It is **not** evaluated as:

```
(10 + 5) * 2
```

Parentheses can explicitly change the order:

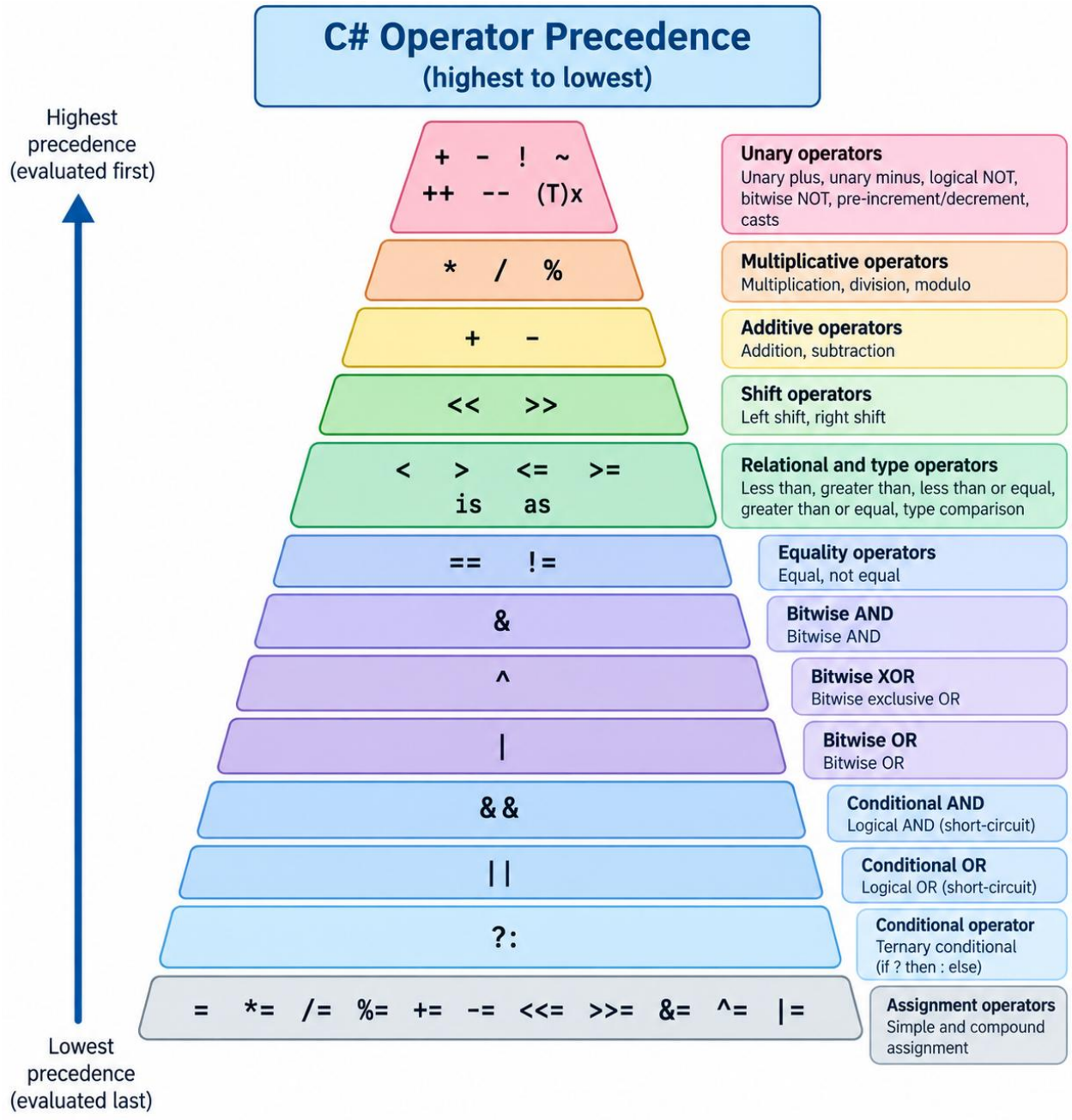
```
int result = (10 + 5) * 2;
```

Now the result is 30.

Tip — Prefer Clarity

Even when you know the precedence rules, parentheses can make complex expressions easier to understand:

```
bool eligible = (age >= 18) && (balance > 0);
```



---

9. Null-Related Operators

C# provides operators that simplify working with values that may be **`null`**.

9.1 Nullable Type `?`

A value type can be made nullable:

```
int? score = null;
```

The variable can contain either:

an **`int`**

OR

`null`

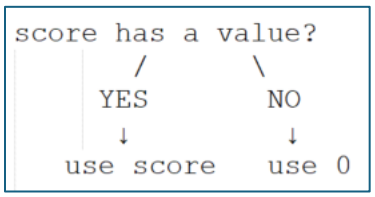
9.2 Null-Coalescing Operator `??`

`??` provides an alternative when the left operand is null.

```
int? score = null;

int displayedScore = score ?? 0;
```

Conceptually:



9.3 Null-Coalescing Assignment `??=`

`??=` assigns a value only when the variable currently evaluates to **`null`**.

```
string? name = null;

name ??= "Unknown";
```

Afterward:

```
name = "Unknown"
```

If **`name`** already contained a value, it would remain unchanged.

9.4 Null-Conditional Member Access `?.`

Suppose an object may be **`null`**:

```
string? name = null;
```

```
int? length = name?.Length;
```

If **`name`** is **`null`**, **`name?.Length`** evaluates to **`null`** rather than accessing **`Length`**.

The corresponding element-access form is:

```
a?[index]
```

which performs conditional element access.

---

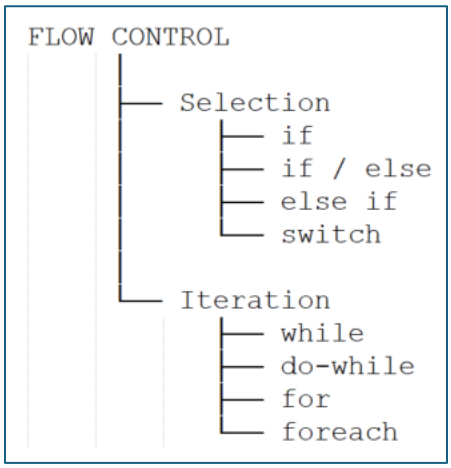
10. Flow Control

Programs normally execute statements sequentially:

Statement 1
↓
Statement 2
↓
Statement 3

Flow-control statements change this sequence.

There are two major categories in this section:



---

11. The `if` Statement

`if` executes code only when its condition evaluates to **`true`**.

Syntax:

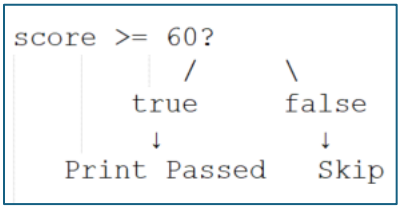
```
if (condition)
```

```
{
    // Executes when condition is true
}
```

Example:

```
int score = 85;
if (score >= 60)
{
    Console.WriteLine("Passed");
}
```

Conceptually:



---

12. `if / else`

`else` provides an alternative path when the **`if`** condition is false.

```
int age = 16;

if (age >= 18)
{
    Console.WriteLine("Adult");
}
else
{
    Console.WriteLine("Minor");
}
```

Exactly one branch executes.

Important

The example in the original slide describes the alternative to `n > 0` as "negative." Technically, 0 would also enter that `else` branch. If three states—negative, zero, and positive—must be distinguished, an `else if` is required.

---

13. Multiple Decisions with `else if`

Use else if when several mutually exclusive conditions need to be evaluated.

```
int score = 82;

if (score >= 90)
{
    Console.WriteLine("Excellent");
}
else if (score >= 75)
{
    Console.WriteLine("Good");
}
else if (score >= 60)
{
    Console.WriteLine("Pass");
}
else
{
    Console.WriteLine("Fail");
}
```

C# checks the conditions **from top to bottom** and stops when it finds the first true condition.

Pitfall — Order Matters

In an `else if` chain, broader conditions placed too early can prevent more specific conditions from ever being reached.

---

14. The switch Statement

A switch selects among several branches based on a value or expression.

```
string domain = "ca";

switch (domain)
{
    case "ca":
        Console.WriteLine("Canada");
        break;

    case "us":
        Console.WriteLine("United States");
        break;

    case "de":
        Console.WriteLine("Germany");
        break;

    default:
        Console.WriteLine("Unknown");
        break;
}
```

Each **`case`** represents a possible branch.

`default` handles values that do not match another case.

The **`break`** statement ends the selected branch.

`if` or `switch`?

A useful guideline is:

```
Complex Boolean conditions
        ↓
       if

One expression with several cases
        ↓
     switch
```

---

15. Adding Conditions with `when`

A **`case`** can include a **`when`** condition to further restrict when that branch matches. The source presents the same **`when`** example twice; it can be consolidated into one explanation.

```
int age = 150;

switch (age)
{
    case var myAge when myAge < 0:
        Console.WriteLine("Age cannot be negative.");
        break;

    case var myAge when myAge > 130:
        Console.WriteLine("This age is very unlikely.");
        break;

    default:
        Console.WriteLine($"Entered age: {age}");
        break;
}
```

Here:

`case var myAge when myAge > 130:`

captures the value in **`myAge`** and executes the case only when the additional condition is true.

---

16. Iteration and Loops

A **loop** repeatedly executes a block of code.

Choosing the appropriate loop depends primarily on what controls repetition:

Loop	Best suited when...
`while`	repetition depends on a condition
`do-while`	the body must execute at least once
`for`	iteration is controlled by a counter
`foreach`	processing each element in a collection

---

17. The `while` Loop

A while loop repeatedly executes while its Boolean condition remains true.

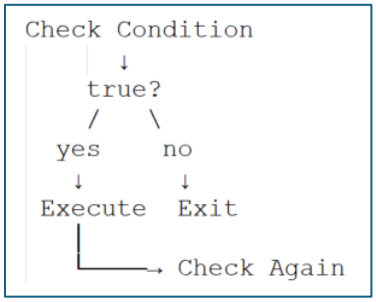
```
int i = 1;

while (i <= 5)
{
    Console.WriteLine(i);
    i++;
}
```

Output:

```
1
2
3
4
5
```

The sequence is:



Warning — Infinite Loops

Something inside the loop must eventually cause the condition to become false. Otherwise, the loop can continue indefinitely.

---

18. The do-while Loop

A **`do-while`** loop checks its condition **after** executing its body. Therefore, its body executes at least once.

```
int count = 0;

do
{
    Console.WriteLine(count);
}
while (count != 0);
```

Although:

`count != 0`

is false, the output still contains:

`0`

because the body executes before the condition is checked.

`while` vs. `do-while`

`while`

`Check → Execute`

`do-while`

`Execute → Check`

This is the essential distinction.

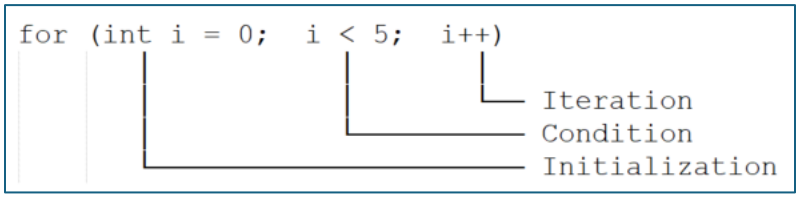
---

19. The `for` Loop

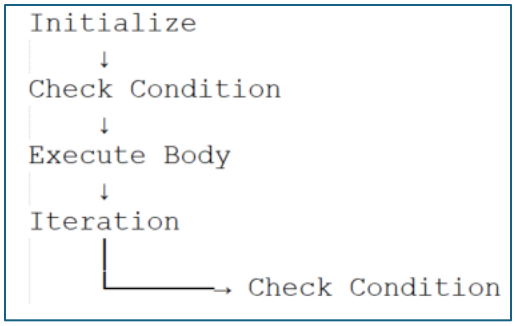
A **`for`** loop combines three important components: initialization, condition, and iteration.

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

The header contains:



Execution proceeds as:



---

20. Multiple Expressions in a `for` Loop

The initialization and iteration sections can contain more than one expression.

For example:

```
for (int i = 0, j = 10; i < 5; i++, j--)
{
    Console.WriteLine($"i = {i}, j = {j}");
}
```

Both `i` and `j` are updated during every iteration.

This demonstrates the general structure shown in the source:

```
for (init1, init2; condition; inc1, inc2)
```

---

21. Nested `for` Loops

A loop can appear inside another loop. The inner loop completes its iterations for every iteration of the outer loop.

```
string[] first = { "A", "B", "C" };
string[] second = { "1", "2" };

for (int i = 0; i < first.Length; i++)
{
    for (int j = 0; j < second.Length; j++)
    {
```

```
            Console.WriteLine(first[i] + second[j]);
        }
}
```

Output:

A1

A2

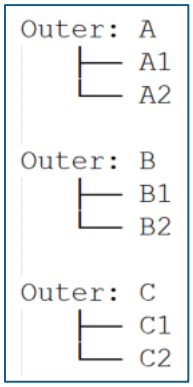
B1

B2

C1

C2

Conceptually:



Note

If the outer loop executes 3 times and the inner loop executes 2 times for each outer iteration, the inner statement executes **3 × 2 = 6** times.

---

22. The `foreach` Loop

foreach simplifies traversal of arrays and other collections. Unlike a typical for loop, it does not require an explicit counter.

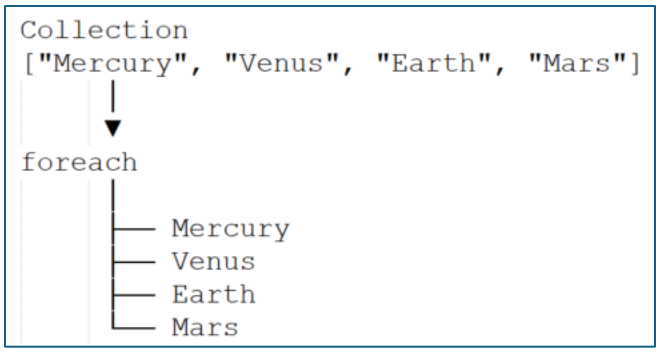
```
string[] planets =
{
    "Mercury",
    "Venus",
    "Earth",
    "Mars"
};

foreach (string planet in planets)
{
    Console.WriteLine(planet);
}
```

During each iteration, the next element is assigned to:

`planet`

Conceptually:



Use **`foreach`** when you primarily need to process each element and do not need to manage the index explicitly.

---

23. `break`

`break` immediately terminates the nearest applicable loop or **`switch`**.

Example:

```
for (int i = 1; i <= 100; i++)
{
    if (i == 5)
    {
        break;
    }

    Console.WriteLine(i);
}
```

Output:

1

2

3

4

When **`i`** becomes **5**, **`break`** terminates the loop.

```
Loop
  ↓
Condition met?
  ├── No  → Continue loop
  └── Yes → break → Exit loop
```

---

24. `continue`

`continue` does **not** terminate the entire loop. Instead, it skips the remainder of the current iteration and proceeds with the next iteration.

Example:

```
for (int i = 1; i <= 10; i++)
{
    if (i % 2 == 0)
    {
        continue;
    }

    Console.WriteLine(i);
}
```

Output:

```
1
3
5
7
9
```

When `i` is even, `continue` skips:

```
Console.WriteLine(i);
```

and moves to the next iteration.

`break` vs. `continue`

```
break
   ↓
Exit the loop completely


continue
   ↓
Skip the rest of this iteration
   ↓
Start the next iteration
```

---

25. Practical Lab — Student Score Analyzer

This short exercise combines the major concepts into one console application without introducing additional topics.

Step 1 — Create the Project

Create a Console App named:

```
ScoreAnalyzer
```

Step 2 — Create Scores

```
int[] scores = { 78, 92, 55, 84, 67, 100 };
```

Step 3 — Process the Scores

Use a **`foreach`** loop:

```
foreach (int score in scores)
{
    if (score >= 90)
    {
        Console.WriteLine($"{score}: Excellent");
    }
    else if (score >= 60)
    {
        Console.WriteLine($"{score}: Pass");
    }
    else
    {
        Console.WriteLine($"{score}: Fail");
    }
}
```

This combines:

- iteration;
- relational operators;
- **`if / else if / else`**;
- string interpolation.

Step 4 — Calculate a Total with Compound Assignment

```
int[] scores = { 85, 90, 78, 92, 88 };
int total = 0;

foreach (int score in scores)
{
    total += score;
}

Console.WriteLine($"Total: {total}");
```

Here:

`total += score;`

is equivalent to:

`total = total + score;`

Step 5 — Use Modulo

Determine whether each score is even or odd:

```
int[] scores = { 85, 90, 78, 92, 88 };
foreach (int score in scores)
{
    if (score % 2 == 0)
    {
        Console.WriteLine($"{score} is even.");
    }
    else
    {
        Console.WriteLine($"{score} is odd.");
    }
}
```

Step 6 — Experiment with `continue`

Skip scores below 60:

```
int[] scores = { 85, 90, 78, 92, 88 };
foreach (int score in scores)
{
    if (score < 60)
    {
        continue;
    }

    Console.WriteLine($"Passing score: {score}");
}
```

Step 7 — Experiment with `break`

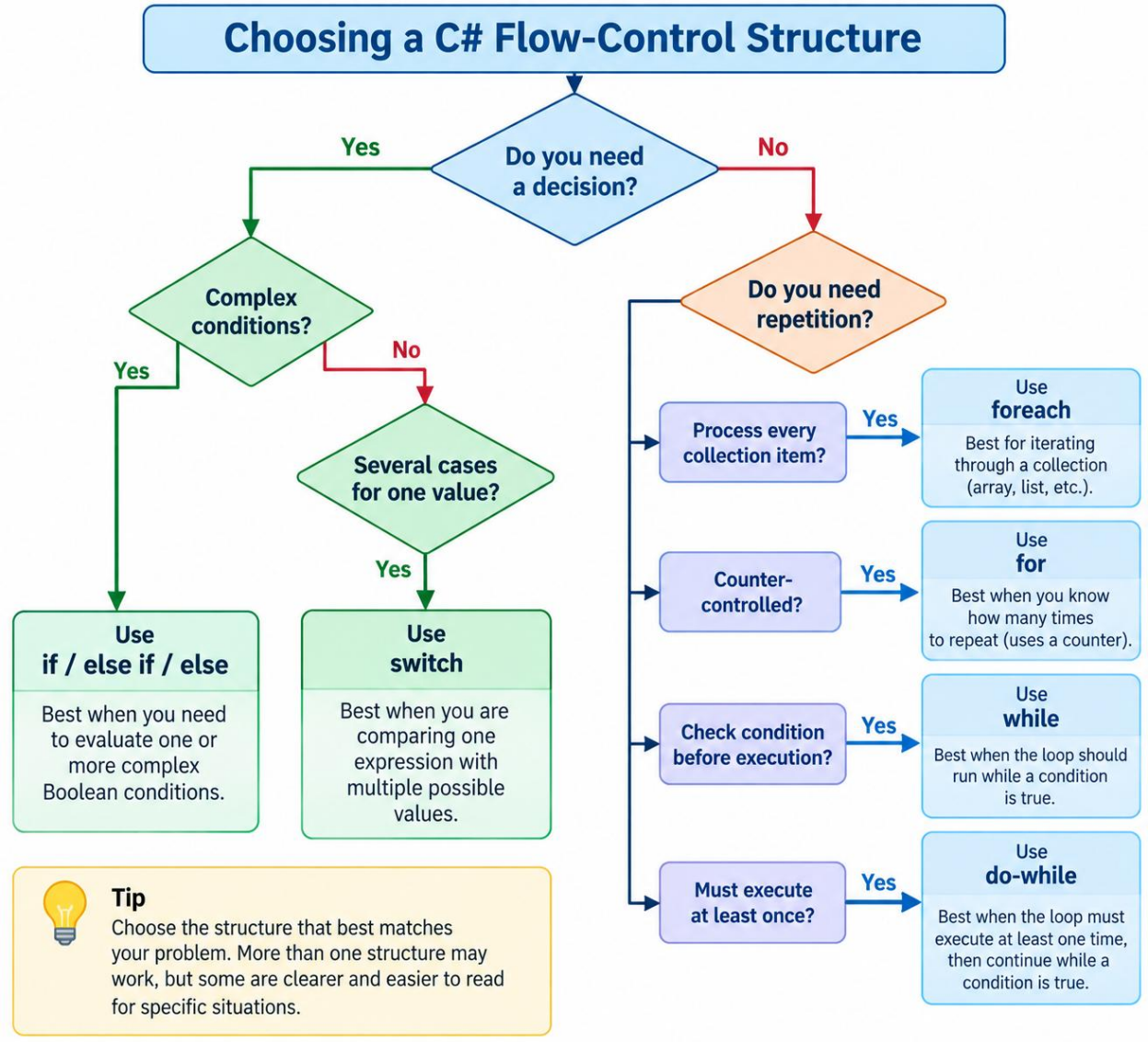
Stop searching when the first perfect score is found:

```
int[] scores = { 85, 90, 78, 92, 88 };
foreach (int score in scores)
{
    if (score == 100)
    {
        Console.WriteLine("Perfect score found.");
        break;
    }
}
```

---

26. Choosing the Appropriate Flow-Control Structure

A useful decision guide is:



27. Key Concepts to Remember

Operators and flow control work together. Relational and logical operators construct the Boolean expressions used by selection and iteration statements.

